

1 July 28, 2025 (Class 1)

Course Logistics

2 July 29, 2025 (Class 2)

Paper reading: Nature Review "Deep Learning", LeCun et al.

3 August 1, 2025 (Class 3)

Continued paper reading: Nature Review "Deep Learning", LeCun et al.

4 August 4, 2025 (Class 4)

4.1 Supervised Learning Setting

4.1.1 Data

Let $\underline{x} \in \mathcal{X}$ be the input data, where $\mathcal{X}$ is the input space, and $y \in \mathcal{Y}$ is the corresponding label, where $\mathcal{Y}$ is the label space. We define

$$\begin{aligned}\mathcal{D}_{train} &= (\underline{x}_i^t, y_i^t)_{i=1}^N \\ \mathcal{D}_{val} &= (\underline{x}_i^v, y_i^v)_{i=1}^M\end{aligned}$$

Setting : $(\underline{x}, y) \in p(\underline{x}, y)$, where p is a fixed but unknown distribution over $\mathcal{X} \times \mathcal{Y}$.

4.1.2 Model/Machine

A parameterized function $f(\underline{x}; \theta)$ that maps an input data point $\underline{x}$ to a label $\hat{y}$.

$$f : \mathcal{X} \times \mathcal{H} \rightarrow \mathcal{Y}$$

where $\mathcal{H}$ is the parameter space, and θ are the parameters of the model.

$$f(\underline{x}; \theta) = f_L^{(\theta_L)} \circ f_{L-1}^{(\theta_{L-1})} \circ \dots \circ f_1^{(\theta_1)}$$

In several settings, we have a template or a building block model.

Multi-Layer Perceptron (MLP)

$$f = \sigma(\underline{W}\underline{x} + \underline{b})$$

where σ is a non-linear activation function, $\underline{W}$ is the weight matrix, and $\underline{b}$ is the bias vector.

Convolutional Neural Network (CNN)

$$f = \sigma(\text{Conv}(\underline{W}, \underline{x}) + \underline{B})$$

where σ is a non-linear activation function, Conv is the convolution operation, $\underline{W}$ is the filter/kernel, and $\underline{B}$ is the bias term.

Transformer

$$f = \text{Transformer}(\underline{\mathbf{x}}, \theta)$$

Recurrent Neural Network (RNN)

$$f^t = \sigma(\underline{\mathbf{W}} \cdot \underline{\mathbf{x}}^t + R \cdot f^{t-1})$$

where σ is a non-linear activation function, $\underline{\mathbf{W}}$ is the weight matrix, R is the recurrent weight matrix, and f^{t-1} is the output from the previous time step.

4.1.3 Optimization

We want to find the parameters θ such that the model

- Low training loss, i.e. on $\mathcal{D}_{train}$
- Generalizes well, i.e. low validation loss on $\mathcal{D}_{val}$

Let per sample distance be defined as

$$d : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$$

Let the loss $\mathcal{L}(\theta)$ be defined as

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N d(f(\underline{\mathbf{x}}_i; \theta), y_i)$$

We want to find the optimal parameters θ^* such that

$$\theta^* = \arg \min_{\theta \in \mathcal{H}} \mathcal{L}(\theta)$$

- 5 August 5, 2025 (Class 5)
- 6 August 8, 2025 (Class 6)
- 7 August 11, 2025 (Class 7)
- 8 August 12, 2025 (Class 8)
- 9 August 18, 2025 (Class 9)
- 10 August 19, 2025 (Class 10)
- 11 August 21, 2025 (Class 11)
- 12 August 25, 2025 (Class 12)
- 13 August 26, 2025 (Class 13)
- 14 August 29, 2025 (Class 14)
- 15 September 1, 2025 (Class 15)
- 16 September 2, 2025 (Class 16)
- 17 September 4, 2025 (Class 17)

17.1 Momentum

The momentum update rule is defined as,

$$\begin{aligned}v &\leftarrow \alpha v - \epsilon \nabla_{\theta} \left(\frac{1}{m} \sum_{i=1}^m L \left(f(x^{(i)}; \theta), y^{(i)} \right) \right) \\ \theta &\leftarrow \theta + v.\end{aligned}$$

Algorithm 8.2 Stochastic gradient descent (SGD) with momentum

Require: Learning rate ϵ , momentum parameter α .

Require: Initial parameter $\boldsymbol{\theta}$, initial velocity $\mathbf{v}$.

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient estimate: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\boldsymbol{\theta}} \sum_i L(f(\mathbf{x}^{(i)}; \boldsymbol{\theta}), \mathbf{y}^{(i)})$

 Compute velocity update: $\mathbf{v} \leftarrow \alpha \mathbf{v} - \epsilon \mathbf{g}$

 Apply update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \mathbf{v}$

end while

17.2 Nesterov Momentum

The Nesterov momentum rule is defined as,

$$\begin{aligned} \mathbf{v} &\leftarrow \alpha \mathbf{v} - \epsilon \nabla_{\boldsymbol{\theta}} \left(\frac{1}{m} \sum_{i=1}^m L \left(f(\mathbf{x}^{(i)}; \boldsymbol{\theta} + \alpha \mathbf{v}), \mathbf{y}^{(i)} \right) \right) \\ \boldsymbol{\theta} &\leftarrow \boldsymbol{\theta} + \mathbf{v}. \end{aligned}$$

18 September 8, 2025 (Class 18)

19 September 9, 2025 (Class 19)

20 AdaGrad (Adaptive Gradient)

Algorithm 8.4 The AdaGrad algorithm

Require: Global learning rate ϵ

Require: Initial parameter θ

Require: Small constant δ , perhaps 10^{-7} , for numerical stability

Initialize gradient accumulation variable $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Accumulate squared gradient: $\mathbf{r} \leftarrow \mathbf{r} + \mathbf{g} \odot \mathbf{g}$

 Compute update: $\Delta\theta \leftarrow -\frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \mathbf{g}$. (Division and square root applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta\theta$

end while

20.1 RMSProp

Algorithm 8.5 The RMSProp algorithm

Require: Global learning rate ϵ , decay rate ρ .

Require: Initial parameter θ

Require: Small constant δ , usually 10^{-6} , used to stabilize division by small numbers.

Initialize accumulation variables $\mathbf{r} = \mathbf{0}$

while stopping criterion not met **do**

 Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

 Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

 Accumulate squared gradient: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \mathbf{g} \odot \mathbf{g}$

 Compute parameter update: $\Delta\theta = -\frac{\epsilon}{\sqrt{\delta + \mathbf{r}}} \odot \mathbf{g}$. ($\frac{1}{\sqrt{\delta + \mathbf{r}}}$ applied element-wise)

 Apply update: $\theta \leftarrow \theta + \Delta\theta$

end while

20.2 Adam (Adaptive Moments)

Algorithm 8.7 The Adam algorithm

Require: Step size ϵ (Suggested default: 0.001)

Require: Exponential decay rates for moment estimates, ρ_1 and ρ_2 in $[0, 1)$.
(Suggested defaults: 0.9 and 0.999 respectively)

Require: Small constant δ used for numerical stabilization. (Suggested default: 10^{-8})

Require: Initial parameters θ

Initialize 1st and 2nd moment variables $\mathbf{s} = \mathbf{0}$, $\mathbf{r} = \mathbf{0}$

Initialize time step $t = 0$

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient: $\mathbf{g} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

$t \leftarrow t + 1$

Update biased first moment estimate: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \mathbf{g}$

Update biased second moment estimate: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \mathbf{g} \odot \mathbf{g}$

Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}$

Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$

Compute update: $\Delta \theta = -\epsilon \frac{\hat{\mathbf{s}}}{\sqrt{\hat{\mathbf{r}} + \delta}}$ (operations applied element-wise)

Apply update: $\theta \leftarrow \theta + \Delta \theta$

end while

20.3 Generative Adversarial Networks

See [Generative Adversarial Networks].

20.4 Variational Autoencoders

See [Variational Autoencoders]

20.4.1 See also

20.4.2 References